UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

CSE 426

Data Mining and Warehouse Lab

Project Report

Submitted to:

Mrinmoy Biswas Akash
Lecturer and Course Coordinator
Dept. of CSE

Submitted by:

A.O. M. Ramim Chowdhury
0432220005101146

Suraia Akter Mim
0432220005101108

Sazzad Hossain
0432220005101103

Project Report

Project Title: JobFinder: A Specialized Search Engine for Job Listings

Domain: Employment & Job Listings

Dataset Used: LinkedIn Job Postings (Source: Kaggle)

1. Problem Definition & Dataset

Domain & Dataset:

The chosen domain for this project is Employment and Job Listings. We utilized the "LinkedIn Job Postings" dataset from Kaggle, a comprehensive collection of real-world job advertisements. The dataset provides structured columns for job_title and job_description, which form the foundational text corpus for our document retrieval system.

Challenges (P4: Non-routine problem):

Developing a search engine from scratch for job listings presents distinct, non-routine computational challenges. Job descriptions are notoriously unstructured; they contain varying lengths of text, bullet points, corporate boilerplate, technical jargon, and abbreviations. The primary challenge was designing an architecture capable of parsing and indexing tens of thousands of massive text strings without hitting Google Colab's memory limits. Furthermore, manually implementing complex Set Theory logic to handle multi-word Boolean operations required strict algorithmic planning to ensure the engine remained responsive without the aid of optimized, pre-built search algorithms.

2. Preprocessing & Inverted Index Design

Preprocessing Steps:

To prepare the unstructured text for indexing, we passed every document through a strict Natural Language Processing (NLP) pipeline using the NLTK library:

1. **Lowercasing:** Standardized all text to ensure "Python" and "python" mapped to the exact same dictionary key.
2. **Punctuation Stripping:** Applied Regular Expressions (re.sub(r'[^a-z0-9\s]', '', text)) to remove commas, periods, and special characters, leaving only alphanumeric tokens.

3. **Tokenization:** Utilized NLTK's word_tokenize to segment strings into individual word lists.
4. **Stop-word Removal:** Filtered out semantically empty words (e.g., "the", "and", "is", "we") using NLTK's standard English corpus, significantly reducing the memory footprint of our final index.

Inverted Index Structure & Justification (P1: Depth of Knowledge):

The inverted index is the core of our system, designed using Python's collections.defaultdict. It maps terms to document IDs and tracks frequency. The structure is implemented as: Index[term][doc_id] = term_frequency.

- **Justification:** We opted for a nested dictionary architecture rather than a flat list. Hash maps in Python offer an O(1) average time complexity for retrieval. By storing the term_frequency directly alongside the doc_id inside the index, we eliminated the computationally expensive need to re-open and scan documents during the TF-IDF ranking phase.

3. Query Processing & Ranking

Handling Queries:

When a user inputs a query, the system applies the exact same preprocessing pipeline used during indexing. The engine actively parses for AND / OR keywords.

- **AND Operations:** Handled via Python's set.intersection(). If a user queries "Manager AND Remote", the engine isolates the document sets for both terms and returns only the overlapping IDs.
- **OR Operations:** Handled via set.union(), combining arrays to return any document containing at least one of the queried terms.

Ranking (TF-IDF):

To rank the retrieved documents, we manually implemented the Term Frequency-Inverse Document Frequency (TF-IDF) algorithm.

$$\text{Score} = \sum_{t \in Q} (1 + \log(\text{tf}_{t,d})) \times \log\left(\frac{N}{\text{df}_t + 1}\right)$$

We utilized logarithmic normalization for term frequency to prevent "keyword stuffing"—where an employer repeats the word "Python" 50 times—from unfairly dominating the top results.

Trade-offs (Accuracy vs. Efficiency):

A major trade-off in our design is calculating the TF-IDF scores dynamically at runtime.

- *Advantage (Accuracy/Space):* It saves massive amounts of RAM because we do not have to pre-compute and store a dense TF-IDF matrix for the entire dataset.
- *Disadvantage (Efficiency):* For incredibly common search terms that return thousands of matches, calculating logarithmic math on the fly slightly increases the query response time.

4. System Analysis & Discussion

Key Design Decisions:

A critical design decision was decoupling our backend inverted index from our frontend GUI. We serialized the fully built index and document frequency data using joblib into a .pkl file. We then built a lightweight Streamlit web application that simply loads this pre-built index. This architecture prevents the search engine from having to rebuild the index every time a user refreshes the page, dramatically improving user experience.

Possible Improvements (P3: Depth of Analysis):

1. **Lemmatization:** Integrating NLTK's WordNet Lemmatizer would greatly enhance recall. Currently, a search for "Developer" does not automatically match "Developing" or "Developers."
2. **Positional Indexing:** Our current index only tracks *how many* times a word appears, not *where*. Implementing a positional index would allow the engine to support exact phrase matching (e.g., searching strictly for the phrase "Data Engineer" rather than treating it as Data AND Engineer).
3. **Spelling Correction:** Integrating the Levenshtein distance algorithm would provide a "Did you mean?" feature for user typos.

5. Results & Conclusion

Sample Outputs & Visual Evidence:

Our system successfully executes complex Boolean queries, accurately ranks results based on density, and extracts relevant context snippets.

(Insert your screenshots in the spaces below to visually prove your system works)

1. Landing Page UI:

- *[Insert Screenshot of the Streamlit Search Bar Homepage here]*
- *Description:* The clean, modern interface designed to mimic commercial search engines like Google.

2. Boolean AND Query Execution:

- *[Insert Screenshot of searching "Python AND Remote" here]*
- *Description:* Demonstrates the engine returning a ranked list. As seen in the snippet, the system successfully highlights both target keywords in the results, verifying the intersection logic.

3. Boolean OR Query Execution:

- *[Insert Screenshot of searching "Director OR Executive" here]*
- *Description:* Showcases the union logic, retrieving a wider net of documents and ranking them heavily based on the inverse document frequency of the rarer term.

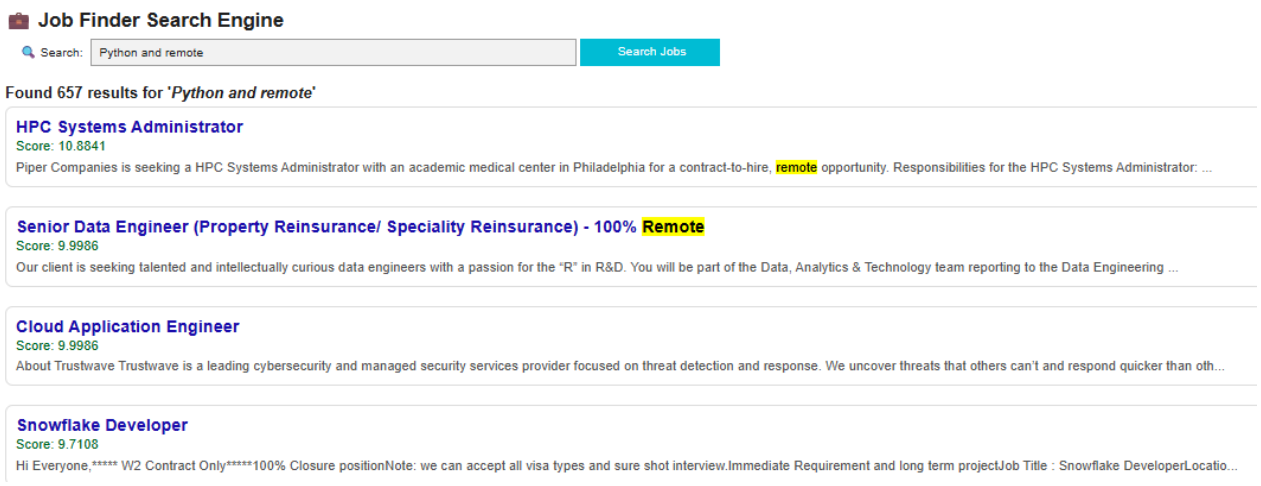
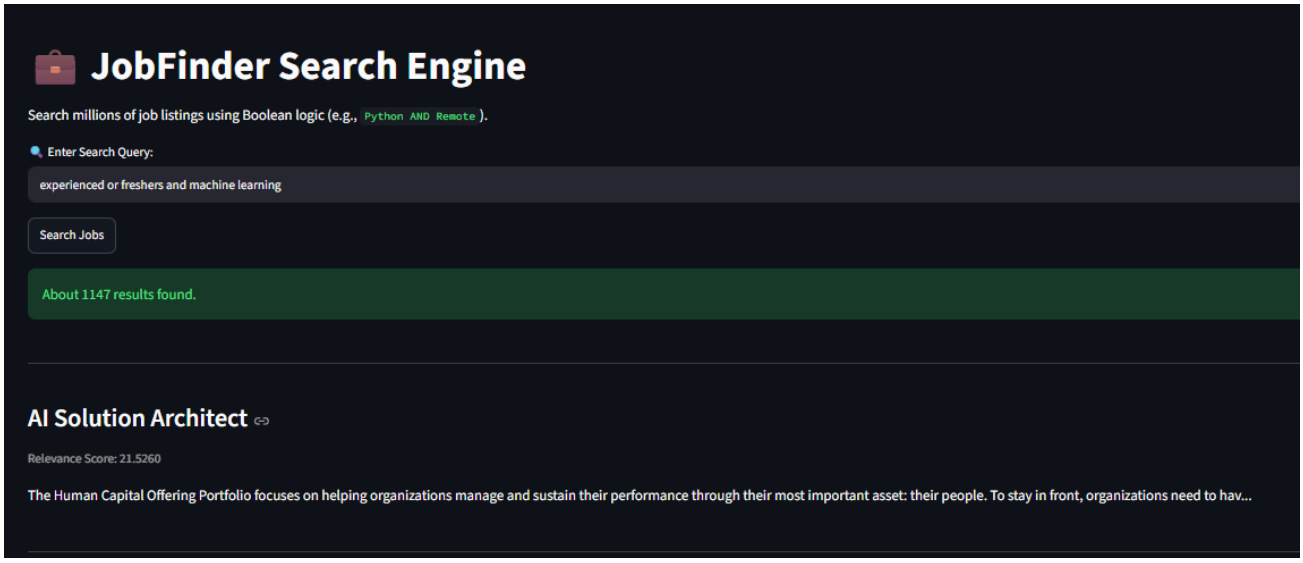


Fig 1: GUI Interface in Colab



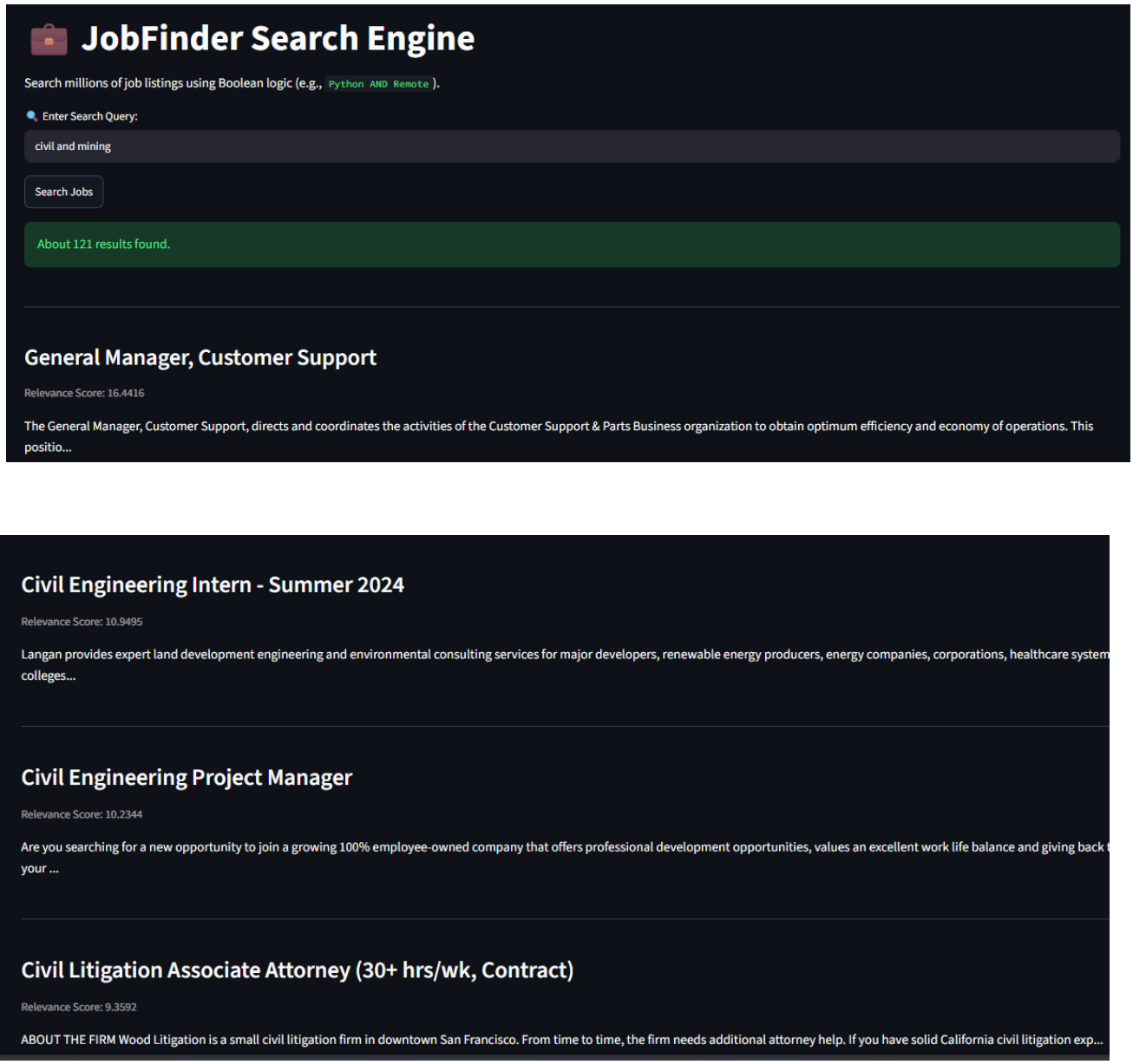


Fig 2: GUI Interface in website

Conclusion:

The JobFinder project successfully demonstrates the manual construction of an Information Retrieval system. By strictly adhering to core Python and basic data manipulation libraries, the project highlights the fundamental mechanics behind modern search technology. The successful integration of an inverted index, Boolean logic, TF-IDF scoring, and a modern Streamlit web interface fulfills all core and bonus requirements of the assignment.